

# Line-by-Line Mathematical Guide for HAETAE\_Assumptions.ec

Generated HAETAE EasyCrypt proof guide

2026-05-11

## File Role

Abstract hardness games and reduction interfaces.

## How To Read This Script

Read this file as the formal trust boundary. It represents MLWE, Module-SIS, and bimodal assumptions as explicit game probabilities.

## Line-by-Line Mathematical Reading

Each row preserves one EasyCrypt source line and gives the intended mathematical or proof-script reading. Blank rows are kept because they mark proof structure in long EasyCrypt developments.

| Line | EasyCrypt source                                                          | Mathematical reading                                                                                                                                        |
|------|---------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1    | <code>require import AllCore Distr List.</code>                           | Imports AllCore Distr List. Mathematically, this exposes earlier carrier sets, operations, games, and lemmas that the current file may cite.                |
| 2    | <code>require import HAETAE_Params HAETAE_Algebra.</code>                 | Imports HAETAE_Params HAETAE_Algebra. Mathematically, this exposes earlier carrier sets, operations, games, and lemmas that the current file may cite.      |
| 3    | <code>blank</code>                                                        | Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.                                                                  |
| 4    | <code>theory HAETAE_Assumptions.</code>                                   | Begins the namespace HAETAE_Assumptions, grouping the declarations as one checked theory.                                                                   |
| 5    | <code>blank</code>                                                        | Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.                                                                  |
| 6    | <code>import HAETAE_Params.</code>                                        | Opens HAETAE_Params so its names can be used without qualification in the following proof.                                                                  |
| 7    | <code>import HAETAE_Algebra.</code>                                       | Opens HAETAE_Algebra so its names can be used without qualification in the following proof.                                                                 |
| 8    | <code>blank</code>                                                        | Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.                                                                  |
| 9    | <code>type mlwe_instance.</code>                                          | Declares carrier set <code>mlwe_instance</code> . EasyCrypt will type-check all later functions and games over this mathematical domain.                    |
| 10   | <code>type module_sis_instance = matrix * polyveck.</code>                | Declares carrier set <code>module_sis_instance</code> . EasyCrypt will type-check all later functions and games over this mathematical domain.              |
| 11   | <code>type module_sis_solution = polyvecl.</code>                         | Declares carrier set <code>module_sis_solution</code> . EasyCrypt will type-check all later functions and games over this mathematical domain.              |
| 12   | <code>type bimodal_selftarget_msis_instance = module_sis_instance.</code> | Declares carrier set <code>bimodal_selftarget_msis_instance</code> . EasyCrypt will type-check all later functions and games over this mathematical domain. |
| 13   | <code>type bimodal_selftarget_msis_solution = module_sis_solution.</code> | Declares carrier set <code>bimodal_selftarget_msis_solution</code> . EasyCrypt will type-check all later functions and games over this mathematical domain. |
| 14   | <code>blank</code>                                                        | Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.                                                                  |

| Line | EasyCrypt source                                                           | Mathematical reading                                                                                                                           |
|------|----------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------|
| 15   | op [lossless] dmlwe_real : mode -> mlwe_instance distr.                    | Defines dmlwe_real as a distribution-valued object; later probability proofs rely on its losslessness, support, or point bounds.               |
| 16   | op [lossless] dmlwe_random : mode -> mlwe_instance distr.                  | Defines dmlwe_random as a distribution-valued object; later probability proofs rely on its losslessness, support, or point bounds.             |
| 17   | op [lossless] dmodule_sis_instance : mode -> module_sis_instance distr.    | Defines dmodule_sis_instance as a distribution-valued object; later probability proofs rely on its losslessness, support, or point bounds.     |
| 18   | op [lossless] dbimodal_selftarget_msis_instance :                          | Defines operation dbimodal_selftarget_msis_instance, a total mathematical function or abbreviation used by later declarations.                 |
| 19   | mode -> bimodal_selftarget_msis_instance distr.                            | Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.                 |
| 20   | blank                                                                      | Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.                                                     |
| 21   | op module_sis_eval                                                         | Defines operation module_sis_eval, a total mathematical function or abbreviation used by later declarations.                                   |
| 22   | (md : mode) (x : module_sis_instance)                                      | Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.                 |
| 23   | (sol : module_sis_solution) : polyveck =                                   | Part of an equality or definition, identifying two mathematical expressions or assigning a program value.                                      |
| 24   | polyveck_sub md (matrix_vec_mul md x.'1 sol) x.'2.                         | Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.                 |
| 25   | blank                                                                      | Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.                                                     |
| 26   | op module_sis_zero_instance (md : mode) : module_sis_instance =            | Defines operation module_sis_zero_instance, a total mathematical function or abbreviation used by later declarations.                          |
| 27   | ([], polyveck_zero md).                                                    | Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.                 |
| 28   | blank                                                                      | Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.                                                     |
| 29   | op module_sis_solution_wf (md : mode) (sol : module_sis_solution) : bool = | Defines predicate module_sis_solution_wf, an event or well-formedness condition used in later implications and game proofs.                    |
| 30   | polyveckl_wf md sol.                                                       | Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.                 |
| 31   | blank                                                                      | Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.                                                     |
| 32   | op module_sis_valid                                                        | Defines operation module_sis_valid, a total mathematical function or abbreviation used by later declarations.                                  |
| 33   | (md : mode) (x : module_sis_instance)                                      | Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.                 |
| 34   | (sol : module_sis_solution) : bool =                                       | Part of an equality or definition, identifying two mathematical expressions or assigning a program value.                                      |
| 35   | module_sis_solution_wf md sol /\                                           | Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.                 |
| 36   | matrix_vec_mul md x.'1 sol = x.'2.                                         | Part of an equality or definition, identifying two mathematical expressions or assigning a program value.                                      |
| 37   | blank                                                                      | Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.                                                     |
| 38   | op bimodal_selftarget_msis_valid                                           | Defines operation bimodal_selftarget_msis_valid, a total mathematical function or abbreviation used by later declarations.                     |
| 39   | (md : mode) (x : bimodal_selftarget_msis_instance)                         | Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.                 |
| 40   | (sol : bimodal_selftarget_msis_solution) : bool =                          | Part of an equality or definition, identifying two mathematical expressions or assigning a program value.                                      |
| 41   | module_sis_valid md x sol.                                                 | Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.                 |
| 42   | blank                                                                      | Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.                                                     |
| 43   | lemma module_sis_eval_zero_instance md sol :                               | States checked lemma module_sis_eval_zero_instance. The following proof script must derive this proposition from prior definitions and lemmas. |
| 44   | module_sis_eval md (module_sis_zero_instance md) sol = polyveck_zero md.   | Part of an equality or definition, identifying two mathematical expressions or assigning a program value.                                      |
| 45   | proof.                                                                     | Begins the proof script for the preceding proposition.                                                                                         |
| 46   | by rewrite /module_sis_eval /module_sis_zero_instance                      | Discharges the current goal in one step using the cited simplification, lemma, or automation.                                                  |
| 47   | /matrix_vec_mul /polyveck_sub /polyveck_add /polyveck_neg.                 | Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.                 |
| 48   | qed.                                                                       | Ends the proof; EasyCrypt has accepted all remaining obligations.                                                                              |

| Line | EasyCrypt source                                                             | Mathematical reading                                                                                                                         |
|------|------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------|
| 49   | <i>blank</i>                                                                 | Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.                                                   |
| 50   | lemma module_sis_zero_valid md :                                             | States checked lemma module_sis_zero_valid. The following proof script must derive this proposition from prior definitions and lemmas.       |
| 51   | module_sis_valid md (module_sis_zero_instance md) (polyvecl_zero md).        | Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.               |
| 52   | proof.                                                                       | Begins the proof script for the preceding proposition.                                                                                       |
| 53   | rewrite /module_sis_valid /module_sis_solution_wf /module_sis_zero_instance. | Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.                         |
| 54   | split.                                                                       | Splits a conjunction or structured goal into independent proof obligations.                                                                  |
| 55   | + by apply polyvecl_zero_wf.                                                 | Solves a proof branch produced by a previous split, transitivity, or case analysis.                                                          |
| 56   | by rewrite /matrix_vec_mul.                                                  | Discharges the current goal in one step using the cited simplification, lemma, or automation.                                                |
| 57   | qed.                                                                         | Ends the proof; EasyCrypt has accepted all remaining obligations.                                                                            |
| 58   | <i>blank</i>                                                                 | Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.                                                   |
| 59   | lemma module_sis_eval_wf md x sol :                                          | States checked lemma module_sis_eval_wf. The following proof script must derive this proposition from prior definitions and lemmas.          |
| 60   | polyveck_wf md (module_sis_eval md x sol).                                   | Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.               |
| 61   | proof. by rewrite /module_sis_eval; apply polyveck_sub_wf. qed.              | Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.               |
| 62   | <i>blank</i>                                                                 | Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.                                                   |
| 63   | lemma module_sis_valid_target_wf md x sol :                                  | States checked lemma module_sis_valid_target_wf. The following proof script must derive this proposition from prior definitions and lemmas.  |
| 64   | module_sis_valid md x sol =>                                                 | Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.                                       |
| 65   | polyveck_wf md x.'2.                                                         | Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.               |
| 66   | proof.                                                                       | Begins the proof script for the preceding proposition.                                                                                       |
| 67   | rewrite /module_sis_valid.                                                   | Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.                         |
| 68   | move=> [_ targetE].                                                          | Introduces universally quantified variables or hypotheses into the proof context.                                                            |
| 69   | by rewrite -targetE; apply matrix_vec_mul_wf.                                | Discharges the current goal in one step using the cited simplification, lemma, or automation.                                                |
| 70   | qed.                                                                         | Ends the proof; EasyCrypt has accepted all remaining obligations.                                                                            |
| 71   | <i>blank</i>                                                                 | Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.                                                   |
| 72   | op bimodal_to_module_sis_instance :                                          | Defines operation bimodal_to_module_sis_instance, a total mathematical function or abbreviation used by later declarations.                  |
| 73   | mode -> bimodal_selftarget_msis_instance -> module_sis_instance =            | Part of an equality or definition, identifying two mathematical expressions or assigning a program value.                                    |
| 74   | fun (_ : mode) x => x.                                                       | Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.                                       |
| 75   | op bimodal_to_module_sis_solution :                                          | Defines operation bimodal_to_module_sis_solution, a total mathematical function or abbreviation used by later declarations.                  |
| 76   | mode -> bimodal_selftarget_msis_instance ->                                  | Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.               |
| 77   | bimodal_selftarget_msis_solution -> module_sis_solution =                    | Part of an equality or definition, identifying two mathematical expressions or assigning a program value.                                    |
| 78   | fun (_ : mode) (_ : bimodal_selftarget_msis_instance) sol => sol.            | Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.                                       |
| 79   | <i>blank</i>                                                                 | Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.                                                   |
| 80   | lemma bimodal_to_module_sis_valid md x sol :                                 | States checked lemma bimodal_to_module_sis_valid. The following proof script must derive this proposition from prior definitions and lemmas. |
| 81   | bimodal_selftarget_msis_valid md x sol =>                                    | Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.                                       |
| 82   | module_sis_valid md                                                          | Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.               |
| 83   | (bimodal_to_module_sis_instance md x)                                        | Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.               |
| 84   | (bimodal_to_module_sis_solution md x sol).                                   | Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.               |

| Line | EasyCrypt source                                                              | Mathematical reading                                                                                                           |
|------|-------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------|
| 85   | <code>proof.</code>                                                           | Begins the proof script for the preceding proposition.                                                                         |
| 86   | <code>by rewrite /bimodal_selftarget_msis_valid</code>                        | Discharges the current goal in one step using the cited simplification, lemma, or automation.                                  |
| 87   | <code>/bimodal_to_module_sis_instance</code>                                  | Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object. |
| 88   | <code>/bimodal_to_module_sis_solution.</code>                                 | Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object. |
| 89   | <code>qed.</code>                                                             | Ends the proof; EasyCrypt has accepted all remaining obligations.                                                              |
| 90   | <code>blank</code>                                                            | Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.                                     |
| 91   | <code>module type MLWE_Distinguisher = {</code>                               | Defines module interface MLWE_Distinguisher. It specifies the procedures that later games or adversaries may call.             |
| 92   | <code>proc distinguish(x : mlwe_instance) : bool</code>                       | Defines procedure distinguish. Its body is a probabilistic program whose result becomes part of a game event.                  |
| 93   | <code>}.</code>                                                               | Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object. |
| 94   | <code>blank</code>                                                            | Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.                                     |
| 95   | <code>module MLWE_Real_Game(B : MLWE_Distinguisher) = {</code>                | Defines module MLWE_Real_Game, a probabilistic program or game used to create events and success probabilities.                |
| 96   | <code>proc main(md : mode) : bool = {</code>                                  | Defines procedure main. Its body is a probabilistic program whose result becomes part of a game event.                         |
| 97   | <code>var x : mlwe_instance;</code>                                           | Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object. |
| 98   | <code>var b : bool;</code>                                                    | Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object. |
| 99   | <code>blank</code>                                                            | Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.                                     |
| 100  | <code>x &lt;\$ dmlwe_real md;</code>                                          | Samples from a distribution. Mathematically this introduces a random variable with the named law.                              |
| 101  | <code>b &lt;@ B.distinguish(x);</code>                                        | Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object. |
| 102  | <code>return b;</code>                                                        | Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object. |
| 103  | <code>}</code>                                                                | Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object. |
| 104  | <code>}.</code>                                                               | Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object. |
| 105  | <code>blank</code>                                                            | Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.                                     |
| 106  | <code>module MLWE_Random_Game(B : MLWE_Distinguisher) = {</code>              | Defines module MLWE_Random_Game, a probabilistic program or game used to create events and success probabilities.              |
| 107  | <code>proc main(md : mode) : bool = {</code>                                  | Defines procedure main. Its body is a probabilistic program whose result becomes part of a game event.                         |
| 108  | <code>var x : mlwe_instance;</code>                                           | Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object. |
| 109  | <code>var b : bool;</code>                                                    | Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object. |
| 110  | <code>blank</code>                                                            | Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.                                     |
| 111  | <code>x &lt;\$ dmlwe_random md;</code>                                        | Samples from a distribution. Mathematically this introduces a random variable with the named law.                              |
| 112  | <code>b &lt;@ B.distinguish(x);</code>                                        | Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object. |
| 113  | <code>return b;</code>                                                        | Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object. |
| 114  | <code>}</code>                                                                | Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object. |
| 115  | <code>}..</code>                                                              | Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object. |
| 116  | <code>blank</code>                                                            | Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.                                     |
| 117  | <code>module type ModuleSIS_Solver = {</code>                                 | Defines module interface ModuleSIS_Solver. It specifies the procedures that later games or adversaries may call.               |
| 118  | <code>proc solve(x : module_sis_instance) : module_sis_solution option</code> | Defines procedure solve. Its body is a probabilistic program whose result becomes part of a game event.                        |

| Line | EasyCrypt source                                                     | Mathematical reading                                                                                                                 |
|------|----------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------|
| 119  | };                                                                   | Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.       |
| 120  | <i>blank</i>                                                         | Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.                                           |
| 121  | module ModuleSIS_Relation_Game(B : ModuleSIS_Solver) = {             | Defines module ModuleSIS_Relation_Game, a probabilistic program or game used to create events and success probabilities.             |
| 122  | proc main(md : mode) : bool = {                                      | Defines procedure main. Its body is a probabilistic program whose result becomes part of a game event.                               |
| 123  | var x : module_sis_instance;                                         | Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.       |
| 124  | var sol : module_sis_solution option;                                | Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.       |
| 125  | <i>blank</i>                                                         | Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.                                           |
| 126  | x <\$ dmodule_sis_instance md;                                       | Samples from a distribution. Mathematically this introduces a random variable with the named law.                                    |
| 127  | sol <@ B.solve(x);                                                   | Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.       |
| 128  | return sol <> None /\ module_sis_valid md x (oget sol);              | Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.       |
| 129  | }                                                                    | Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.       |
| 130  | };                                                                   | Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.       |
| 131  | <i>blank</i>                                                         | Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.                                           |
| 132  | module type BimodalSelfTargetMSIS_Solver = {                         | Defines module interface BimodalSelfTargetMSIS_Solver. It specifies the procedures that later games or adversaries may call.         |
| 133  | proc solve(x : bimodal_selftarget_msis_instance) :                   | Defines procedure solve. Its body is a probabilistic program whose result becomes part of a game event.                              |
| 134  | bimodal_selftarget_msis_solution option                              | Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.       |
| 135  | };                                                                   | Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.       |
| 136  | <i>blank</i>                                                         | Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.                                           |
| 137  | module BimodalSelfTargetMSIS_Relation_Game(                          | Defines module BimodalSelfTargetMSIS_Relation_Game, a probabilistic program or game used to create events and success probabilities. |
| 138  | B : BimodalSelfTargetMSIS_Solver                                     | Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.       |
| 139  | ) = {                                                                | Part of an equality or definition, identifying two mathematical expressions or assigning a program value.                            |
| 140  | proc main(md : mode) : bool = {                                      | Defines procedure main. Its body is a probabilistic program whose result becomes part of a game event.                               |
| 141  | var x : bimodal_selftarget_msis_instance;                            | Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.       |
| 142  | var sol : bimodal_selftarget_msis_solution option;                   | Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.       |
| 143  | <i>blank</i>                                                         | Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.                                           |
| 144  | x <\$ dbimodal_selftarget_msis_instance md;                          | Samples from a distribution. Mathematically this introduces a random variable with the named law.                                    |
| 145  | sol <@ B.solve(x);                                                   | Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.       |
| 146  | return sol <> None /\ bimodal_selftarget_msis_valid md x (oget sol); | Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.       |
| 147  | }                                                                    | Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.       |
| 148  | };                                                                   | Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.       |
| 149  | <i>blank</i>                                                         | Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.                                           |
| 150  | op lift_bimodal_solution_option                                      | Defines operation lift_bimodal_solution_option, a total mathematical function or abbreviation used by later declarations.            |
| 151  | (md : mode) (x : bimodal_selftarget_msis_instance)                   | Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.       |

| Line | EasyCrypt source                                                              | Mathematical reading                                                                                                                               |
|------|-------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------|
| 152  | (sol : bimodal_selftarget_msis_solution option) :                             | Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.                     |
| 153  | module_sis_solution option =                                                  | Part of an equality or definition, identifying two mathematical expressions or assigning a program value.                                          |
| 154  | if sol = None then None                                                       | Splits proof obligations according to a conditional branch in the program or formula.                                                              |
| 155  | else Some (bimodal_to_module_sis_solution md x (oget sol)).                   | Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.                     |
| 156  | blank                                                                         | Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.                                                         |
| 157  | op dmodule_sis_from_bimodal (md : mode) : module_sis_instance distr =         | Defines dmodule_sis_from_bimodal as a distribution-valued object; later probability proofs rely on its losslessness, support, or point bounds.     |
| 158  | dmap (dbimodal_selftarget_msis_instance md)                                   | Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.                     |
| 159  | (bimodal_to_module_sis_instance md).                                          | Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.                     |
| 160  | blank                                                                         | Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.                                                         |
| 161  | lemma dmodule_sis_from_bimodalE md :                                          | States checked lemma dmodule_sis_from_bimodalE. The following proof script must derive this proposition from prior definitions and lemmas.         |
| 162  | dmodule_sis_from_bimodal md = dbimodal_selftarget_msis_instance md.           | Part of an equality or definition, identifying two mathematical expressions or assigning a program value.                                          |
| 163  | proof.                                                                        | Begins the proof script for the preceding proposition.                                                                                             |
| 164  | by rewrite /dmodule_sis_from_bimodal /bimodal_to_module_sis_instance dmap_id. | Discharges the current goal in one step using the cited simplification, lemma, or automation.                                                      |
| 165  | qed.                                                                          | Ends the proof; EasyCrypt has accepted all remaining obligations.                                                                                  |
| 166  | blank                                                                         | Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.                                                         |
| 167  | lemma dmodule_sis_from_bimodal_lossless md :                                  | States checked lemma dmodule_sis_from_bimodal_lossless. The following proof script must derive this proposition from prior definitions and lemmas. |
| 168  | is_lossless (dmodule_sis_from_bimodal md).                                    | Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.                     |
| 169  | proof.                                                                        | Begins the proof script for the preceding proposition.                                                                                             |
| 170  | rewrite dmodule_sis_from_bimodalE.                                            | Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.                               |
| 171  | by apply dbimodal_selftarget_msis_instance_ll.                                | Discharges the current goal in one step using the cited simplification, lemma, or automation.                                                      |
| 172  | qed.                                                                          | Ends the proof; EasyCrypt has accepted all remaining obligations.                                                                                  |
| 173  | blank                                                                         | Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.                                                         |
| 174  | module BimodalToModuleSIS_Lift_Game(                                          | Defines module BimodalToModuleSIS_Lift_Game, a probabilistic program or game used to create events and success probabilities.                      |
| 175  | B : BimodalSelfTargetMSIS_Solver                                              | Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.                     |
| 176  | ) = {                                                                         | Part of an equality or definition, identifying two mathematical expressions or assigning a program value.                                          |
| 177  | proc main(md : mode) : bool = {                                               | Defines procedure main. Its body is a probabilistic program whose result becomes part of a game event.                                             |
| 178  | var x : bimodal_selftarget_msis_instance;                                     | Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.                     |
| 179  | var lifted_x : module_sis_instance;                                           | Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.                     |
| 180  | var sol : bimodal_selftarget_msis_solution option;                            | Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.                     |
| 181  | var lifted_sol : module_sis_solution;                                         | Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.                     |
| 182  | blank                                                                         | Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.                                                         |
| 183  | x <\$ dbimodal_selftarget_msis_instance md;                                   | Samples from a distribution. Mathematically this introduces a random variable with the named law.                                                  |
| 184  | sol <@ B.solve(x);                                                            | Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.                     |
| 185  | lifted_x <- bimodal_to_module_sis_instance md x;                              | Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.                     |
| 186  | lifted_sol <- bimodal_to_module_sis_solution md x (oget sol);                 | Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.                     |

| Line | EasyCrypt source                                                                                | Mathematical reading                                                                                                                                     |
|------|-------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------|
| 187  | <code>return sol &lt;&gt; None /\ module_sis_valid md lifted_x lifted_sol;</code>               | Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.                           |
| 188  | <code>}</code>                                                                                  | Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.                           |
| 189  | <code>}.</code>                                                                                 | Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.                           |
| 190  | <code>blank</code>                                                                              | Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.                                                               |
| 191  | <code>lemma bimodal_lift_successE md x sol :</code>                                             | States checked lemma <code>bimodal_lift_successE</code> . The following proof script must derive this proposition from prior definitions and lemmas.     |
| 192  | <code>(sol &lt;&gt; None /\ bimodal_selftarget_msis_valid md x (oget sol)) =</code>             | Part of an equality or definition, identifying two mathematical expressions or assigning a program value.                                                |
| 193  | <code>(lift_bimodal_solution_option md x sol &lt;&gt; None /\</code>                            | Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.                           |
| 194  | <code>module_sis_valid md</code>                                                                | Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.                           |
| 195  | <code>(bimodal_to_module_sis_instance md x)</code>                                              | Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.                           |
| 196  | <code>(oget (lift_bimodal_solution_option md x sol)))</code> .                                  | Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.                           |
| 197  | <code>proof.</code>                                                                             | Begins the proof script for the preceding proposition.                                                                                                   |
| 198  | <code>case: (sol = None) =&gt; sol_none.</code>                                                 | Performs case analysis or induction, splitting the mathematical proof by constructors or list structure.                                                 |
| 199  | <code>+ by rewrite /lift_bimodal_solution_option sol_none.</code>                               | Solves a proof branch produced by a previous split, transitivity, or case analysis.                                                                      |
| 200  | <code>+ by rewrite /lift_bimodal_solution_option sol_none /bimodal_selftarget_msis_valid</code> | Solves a proof branch produced by a previous split, transitivity, or case analysis.                                                                      |
| 201  | <code>/bimodal_to_module_sis_instance</code>                                                    | Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.                           |
| 202  | <code>/bimodal_to_module_sis_solution.</code>                                                   | Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.                           |
| 203  | <code>qed.</code>                                                                               | Ends the proof; EasyCrypt has accepted all remaining obligations.                                                                                        |
| 204  | <code>blank</code>                                                                              | Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.                                                               |
| 205  | <code>section BimodalSolverLift.</code>                                                         | Starts section <code>BimodalSolverLift</code> , a scoped region for related declarations and hypotheses.                                                 |
| 206  | <code>blank</code>                                                                              | Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.                                                               |
| 207  | <code>declare module B &lt;: BimodalSelfTargetMSIS_Solver.</code>                               | Declares an abstract module parameter. Mathematically this quantifies over an oracle, adversary, or reduction satisfying the stated interface.           |
| 208  | <code>blank</code>                                                                              | Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.                                                               |
| 209  | <code>lemma bimodal_solver_lift_exact &amp;m md :</code>                                        | States checked lemma <code>bimodal_solver_lift_exact</code> . The following proof script must derive this proposition from prior definitions and lemmas. |
| 210  | <code>Pr[BimodalSelfTargetMSIS_Relation_Game(B).main(md) @ &amp;m : res] =</code>               | Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.                                              |
| 211  | <code>Pr[BimodalToModuleSIS_Lift_Game(B).main(md) @ &amp;m : res].</code>                       | Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.                                              |
| 212  | <code>proof.</code>                                                                             | Begins the proof script for the preceding proposition.                                                                                                   |
| 213  | <code>byequiv (: = {glob B, md} ==&gt; {res}) =&gt; //.</code>                                  | Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.                                                   |
| 214  | <code>proc.</code>                                                                              | Enters procedure-body proof mode for a module procedure.                                                                                                 |
| 215  | <code>wp.</code>                                                                                | Applies weakest-precondition reasoning to a program fragment.                                                                                            |
| 216  | <code>call (: = {glob B, x} ==&gt; {glob B, res}).</code>                                       | Uses a procedure specification or adversary call rule inside a program logic proof.                                                                      |
| 217  | <code>+ by sim.</code>                                                                          | Solves a proof branch produced by a previous split, transitivity, or case analysis.                                                                      |
| 218  | <code>wp.</code>                                                                                | Applies weakest-precondition reasoning to a program fragment.                                                                                            |
| 219  | <code>rnd.</code>                                                                               | Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.                           |
| 220  | <code>by auto =&gt; /&gt;; rewrite /bimodal_selftarget_msis_valid</code>                        | Discharges the current goal in one step using the cited simplification, lemma, or automation.                                                            |
| 221  | <code>/bimodal_to_module_sis_instance</code>                                                    | Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.                           |
| 222  | <code>/bimodal_to_module_sis_solution.</code>                                                   | Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.                           |

| Line | EasyCrypt source                                              | Mathematical reading                                                                                                                     |
|------|---------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------|
| 223  | qed.                                                          | Ends the proof; EasyCrypt has accepted all remaining obligations.                                                                        |
| 224  | blank                                                         | Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.                                               |
| 225  | end section BimodalSolverLift.                                | Closes the current section, discharging local module and variable scope.                                                                 |
| 226  | blank                                                         | Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.                                               |
| 227  | module type ModuleSIS_ModeSolver = {                          | Defines module interface ModuleSIS_ModeSolver. It specifies the procedures that later games or adversaries may call.                     |
| 228  | proc solve(md : mode, x : module_sis_instance) :              | Defines procedure solve. Its body is a probabilistic program whose result becomes part of a game event.                                  |
| 229  | module_sis_solution option                                    | Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.           |
| 230  | }.                                                            | Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.           |
| 231  | blank                                                         | Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.                                               |
| 232  | module type BimodalSelfTargetMSIS_ModeSolver = {              | Defines module interface BimodalSelfTargetMSIS_ModeSolver. It specifies the procedures that later games or adversaries may call.         |
| 233  | proc solve(md : mode, x : bimodal_selftarget_msis_instance) : | Defines procedure solve. Its body is a probabilistic program whose result becomes part of a game event.                                  |
| 234  | bimodal_selftarget_msis_solution option                       | Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.           |
| 235  | }.                                                            | Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.           |
| 236  | blank                                                         | Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.                                               |
| 237  | module ModuleSIS_ModeRelation_Game(                           | Defines module ModuleSIS_ModeRelation_Game, a probabilistic program or game used to create events and success probabilities.             |
| 238  | B : ModuleSIS_ModeSolver                                      | Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.           |
| 239  | ) = {                                                         | Part of an equality or definition, identifying two mathematical expressions or assigning a program value.                                |
| 240  | proc main(md : mode) : bool = {                               | Defines procedure main. Its body is a probabilistic program whose result becomes part of a game event.                                   |
| 241  | var x : module_sis_instance;                                  | Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.           |
| 242  | var sol : module_sis_solution option;                         | Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.           |
| 243  | blank                                                         | Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.                                               |
| 244  | x <\$ dmodule_sis_instance md;                                | Samples from a distribution. Mathematically this introduces a random variable with the named law.                                        |
| 245  | sol <@ B.solve(md, x);                                        | Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.           |
| 246  | return sol <> None /\ module_sis_valid md x (oget sol);       | Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.           |
| 247  | }                                                             | Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.           |
| 248  | }.                                                            | Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.           |
| 249  | blank                                                         | Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.                                               |
| 250  | module BimodalSelfTargetMSIS_ModeRelation_Game(               | Defines module BimodalSelfTargetMSIS_ModeRelation_Game, a probabilistic program or game used to create events and success probabilities. |
| 251  | B : BimodalSelfTargetMSIS_ModeSolver                          | Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.           |
| 252  | ) = {                                                         | Part of an equality or definition, identifying two mathematical expressions or assigning a program value.                                |
| 253  | proc main(md : mode) : bool = {                               | Defines procedure main. Its body is a probabilistic program whose result becomes part of a game event.                                   |
| 254  | var x : bimodal_selftarget_msis_instance;                     | Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.           |
| 255  | var sol : bimodal_selftarget_msis_solution option;            | Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.           |
| 256  | blank                                                         | Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.                                               |



| Line | EasyCrypt source                                                                        | Mathematical reading                                                                                                                                      |
|------|-----------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------|
| 257  | <code>x &lt;\$ dbimodal_selftarget_msis_instance md;</code>                             | Samples from a distribution. Mathematically this introduces a random variable with the named law.                                                         |
| 258  | <code>sol &lt;@ B.solve(md, x);</code>                                                  | Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.                            |
| 259  | <code>return sol &lt;&gt; None /\ bimodal_selftarget_msis_valid md x (oget sol);</code> | Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.                            |
| 260  | <code>}</code>                                                                          | Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.                            |
| 261  | <code>}.</code>                                                                         | Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.                            |
| 262  | <code>blank</code>                                                                      | Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.                                                                |
| 263  | <code>module ModuleSIS_LiftedDistribution_Relation_Game(</code>                         | Defines module <code>ModuleSIS_LiftedDistribution_Relation_Game</code> , a probabilistic program or game used to create events and success probabilities. |
| 264  | <code>B : ModuleSIS_ModeSolver</code>                                                   | Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.                            |
| 265  | <code>) = {</code>                                                                      | Part of an equality or definition, identifying two mathematical expressions or assigning a program value.                                                 |
| 266  | <code>proc main(md : mode) : bool = {</code>                                            | Defines procedure <code>main</code> . Its body is a probabilistic program whose result becomes part of a game event.                                      |
| 267  | <code>var x : module_sis_instance;</code>                                               | Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.                            |
| 268  | <code>var sol : module_sis_solution option;</code>                                      | Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.                            |
| 269  | <code>blank</code>                                                                      | Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.                                                                |
| 270  | <code>x &lt;\$ dbimodal_selftarget_msis_instance md;</code>                             | Samples from a distribution. Mathematically this introduces a random variable with the named law.                                                         |
| 271  | <code>sol &lt;@ B.solve(md, x);</code>                                                  | Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.                            |
| 272  | <code>return sol &lt;&gt; None /\ module_sis_valid md x (oget sol);</code>              | Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.                            |
| 273  | <code>}</code>                                                                          | Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.                            |
| 274  | <code>}.</code>                                                                         | Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.                            |
| 275  | <code>blank</code>                                                                      | Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.                                                                |
| 276  | <code>module BimodalModeSolver_As_ModuleSIS(</code>                                     | Defines module <code>BimodalModeSolver_As_ModuleSIS</code> , a probabilistic program or game used to create events and success probabilities.             |
| 277  | <code>B : BimodalSelfTargetMSIS_ModeSolver</code>                                       | Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.                            |
| 278  | <code>) = {</code>                                                                      | Part of an equality or definition, identifying two mathematical expressions or assigning a program value.                                                 |
| 279  | <code>proc solve(md : mode, x : module_sis_instance) :</code>                           | Defines procedure <code>solve</code> . Its body is a probabilistic program whose result becomes part of a game event.                                     |
| 280  | <code>module_sis_solution option = {</code>                                             | Part of an equality or definition, identifying two mathematical expressions or assigning a program value.                                                 |
| 281  | <code>var sol : bimodal_selftarget_msis_solution option;</code>                         | Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.                            |
| 282  | <code>blank</code>                                                                      | Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.                                                                |
| 283  | <code>sol &lt;@ B.solve(md, x);</code>                                                  | Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.                            |
| 284  | <code>return sol;</code>                                                                | Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.                            |
| 285  | <code>}</code>                                                                          | Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.                            |
| 286  | <code>}.</code>                                                                         | Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.                            |
| 287  | <code>blank</code>                                                                      | Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.                                                                |
| 288  | <code>section BimodalModeSolverLiftedDistribution.</code>                               | Starts section <code>BimodalModeSolverLiftedDistribution</code> , a scoped region for related declarations and hypotheses.                                |
| 289  | <code>blank</code>                                                                      | Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.                                                                |

| Line | EasyCrypt source                                                                      | Mathematical reading                                                                                                                                                         |
|------|---------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 290  | <code>declare module B &lt;: BimodalSelfTargetMSIS_ModeSolver.</code>                 | Declares an abstract module parameter. Mathematically this quantifies over an oracle, adversary, or reduction satisfying the stated interface.                               |
| 291  | <code>blank</code>                                                                    | Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.                                                                                   |
| 292  | <code>lemma bimodal_mode_solver_lifted_distribution_exact &amp;m md :</code>          | States checked lemma <code>bimodal_mode_solver_lifted_distribution_exact</code> . The following proof script must derive this proposition from prior definitions and lemmas. |
| 293  | <code>Pr[BimodalSelfTargetMSIS_ModeRelation_Game(B).main(md) @ &amp;m : res] =</code> | Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.                                                                  |
| 294  | <code>Pr[ModuleSIS_LiftedDistribution_Relation_Game(</code>                           | Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.                                                                  |
| 295  | <code>BimodalModeSolver_As_ModuleSIS(B)).main(md) @ &amp;m : res].</code>             | Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.                                               |
| 296  | <code>proof.</code>                                                                   | Begins the proof script for the preceding proposition.                                                                                                                       |
| 297  | <code>byequiv (: = {glob B, md} ==&gt; {res}) =&gt; //.</code>                        | Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.                                                                       |
| 298  | <code>proc.</code>                                                                    | Enters procedure-body proof mode for a module procedure.                                                                                                                     |
| 299  | <code>inline BimodalModeSolver_As_ModuleSIS(B).solve.</code>                          | Unfolds a procedure body so the proof can reason about its concrete commands.                                                                                                |
| 300  | <code>wp.</code>                                                                      | Applies weakest-precondition reasoning to a program fragment.                                                                                                                |
| 301  | <code>call (: = {glob B, md, x} ==&gt; {glob B, res}).</code>                         | Uses a procedure specification or adversary call rule inside a program logic proof.                                                                                          |
| 302  | <code>+ by sim.</code>                                                                | Solves a proof branch produced by a previous split, transitivity, or case analysis.                                                                                          |
| 303  | <code>wp.</code>                                                                      | Applies weakest-precondition reasoning to a program fragment.                                                                                                                |
| 304  | <code>rnd.</code>                                                                     | Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.                                               |
| 305  | <code>by auto =&gt; /&gt;; rewrite /bimodal_selftarget_msis_valid.</code>             | Discharges the current goal in one step using the cited simplification, lemma, or automation.                                                                                |
| 306  | <code>qed.</code>                                                                     | Ends the proof; EasyCrypt has accepted all remaining obligations.                                                                                                            |
| 307  | <code>blank</code>                                                                    | Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.                                                                                   |
| 308  | <code>end section BimodalModeSolverLiftedDistribution.</code>                         | Closes the current section, discharging local module and variable scope.                                                                                                     |
| 309  | <code>blank</code>                                                                    | Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.                                                                                   |
| 310  | <code>module type MLWE_Reduction = {</code>                                           | Defines module interface <code>MLWE_Reduction</code> . It specifies the procedures that later games or adversaries may call.                                                 |
| 311  | <code>proc main() : bool</code>                                                       | Defines procedure <code>main</code> . Its body is a probabilistic program whose result becomes part of a game event.                                                         |
| 312  | <code>}.</code>                                                                       | Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.                                               |
| 313  | <code>blank</code>                                                                    | Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.                                                                                   |
| 314  | <code>module type BimodalSelfTargetMSIS_Reduction = {</code>                          | Defines module interface <code>BimodalSelfTargetMSIS_Reduction</code> . It specifies the procedures that later games or adversaries may call.                                |
| 315  | <code>proc main() : bool</code>                                                       | Defines procedure <code>main</code> . Its body is a probabilistic program whose result becomes part of a game event.                                                         |
| 316  | <code>}..</code>                                                                      | Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.                                               |
| 317  | <code>blank</code>                                                                    | Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.                                                                                   |
| 318  | <code>module type ModuleSIS_Reduction = {</code>                                      | Defines module interface <code>ModuleSIS_Reduction</code> . It specifies the procedures that later games or adversaries may call.                                            |
| 319  | <code>proc main() : bool</code>                                                       | Defines procedure <code>main</code> . Its body is a probabilistic program whose result becomes part of a game event.                                                         |
| 320  | <code>}..</code>                                                                      | Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.                                               |
| 321  | <code>blank</code>                                                                    | Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.                                                                                   |
| 322  | <code>module MLWE_Game(B : MLWE_Reduction) = {</code>                                 | Defines module <code>MLWE_Game</code> , a probabilistic program or game used to create events and success probabilities.                                                     |
| 323  | <code>proc main() : bool = {</code>                                                   | Defines procedure <code>main</code> . Its body is a probabilistic program whose result becomes part of a game event.                                                         |
| 324  | <code>var b : bool;</code>                                                            | Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.                                               |
| 325  | <code>blank</code>                                                                    | Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.                                                                                   |

| Line | EasyCrypt source                                                                        | Mathematical reading                                                                                                                      |
|------|-----------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------|
| 326  | <code>b &lt;@ B.main();</code>                                                          | Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.            |
| 327  | <code>return b;</code>                                                                  | Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.            |
| 328  | <code>}</code>                                                                          | Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.            |
| 329  | <code>}.</code>                                                                         | Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.            |
| 330  | <code>blank</code>                                                                      | Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.                                                |
| 331  | <code>module BimodalSelfTargetMSIS_Game(B : BimodalSelfTargetMSIS_Reduction) = {</code> | Defines module <code>BimodalSelfTargetMSIS_Game</code> , a probabilistic program or game used to create events and success probabilities. |
| 332  | <code>proc main() : bool = {</code>                                                     | Defines procedure <code>main</code> . Its body is a probabilistic program whose result becomes part of a game event.                      |
| 333  | <code>var b : bool;</code>                                                              | Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.            |
| 334  | <code>blank</code>                                                                      | Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.                                                |
| 335  | <code>b &lt;@ B.main();</code>                                                          | Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.            |
| 336  | <code>return b;</code>                                                                  | Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.            |
| 337  | <code>}</code>                                                                          | Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.            |
| 338  | <code>}..</code>                                                                        | Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.            |
| 339  | <code>blank</code>                                                                      | Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.                                                |
| 340  | <code>module ModuleSIS_Game(B : ModuleSIS_Reduction) = {</code>                         | Defines module <code>ModuleSIS_Game</code> , a probabilistic program or game used to create events and success probabilities.             |
| 341  | <code>proc main() : bool = {</code>                                                     | Defines procedure <code>main</code> . Its body is a probabilistic program whose result becomes part of a game event.                      |
| 342  | <code>var b : bool;</code>                                                              | Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.            |
| 343  | <code>blank</code>                                                                      | Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.                                                |
| 344  | <code>b &lt;@ B.main();</code>                                                          | Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.            |
| 345  | <code>return b;</code>                                                                  | Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.            |
| 346  | <code>}</code>                                                                          | Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.            |
| 347  | <code>}..</code>                                                                        | Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.            |
| 348  | <code>blank</code>                                                                      | Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.                                                |
| 349  | <code>module BimodalMSIS_As_ModuleSIS(B : BimodalSelfTargetMSIS_Reduction) = {</code>   | Defines module <code>BimodalMSIS_As_ModuleSIS</code> , a probabilistic program or game used to create events and success probabilities.   |
| 350  | <code>proc main() : bool = {</code>                                                     | Defines procedure <code>main</code> . Its body is a probabilistic program whose result becomes part of a game event.                      |
| 351  | <code>var b : bool;</code>                                                              | Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.            |
| 352  | <code>blank</code>                                                                      | Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.                                                |
| 353  | <code>b &lt;@ B.main();</code>                                                          | Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.            |
| 354  | <code>return b;</code>                                                                  | Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.            |
| 355  | <code>}</code>                                                                          | Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.            |
| 356  | <code>}..</code>                                                                        | Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.            |
| 357  | <code>blank</code>                                                                      | Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.                                                |
| 358  | <code>section BimodalModuleSISReduction.</code>                                         | Starts section <code>BimodalModuleSISReduction</code> , a scoped region for related declarations and hypotheses.                          |

| Line | EasyCrypt source                                                   | Mathematical reading                                                                                                                              |
|------|--------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------|
| 359  | <i>blank</i>                                                       | Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.                                                        |
| 360  | declare module B <: BimodalSelfTargetMSIS_Reduction.               | Declares an abstract module parameter. Mathematically this quantifies over an oracle, adversary, or reduction satisfying the stated interface.    |
| 361  | <i>blank</i>                                                       | Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.                                                        |
| 362  | lemma bimodal_msis_as_module_sis_exact &m :                        | States checked lemma bimodal_msis_as_module_sis_exact. The following proof script must derive this proposition from prior definitions and lemmas. |
| 363  | Pr[BimodalSelfTargetMSIS_Game(B).main() @ &m : res] =              | Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.                                       |
| 364  | Pr[ModuleSIS_Game(BimodalMSIS_As_ModuleSIS(B)).main() @ &m : res]. | Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.                                       |
| 365  | proof.                                                             | Begins the proof script for the preceding proposition.                                                                                            |
| 366  | byequiv (: = {glob B} ==> = {res}) => //.                          | Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.                                            |
| 367  | proc.                                                              | Enters procedure-body proof mode for a module procedure.                                                                                          |
| 368  | inline BimodalMSIS_As_ModuleSIS(B).main.                           | Unfolds a procedure body so the proof can reason about its concrete commands.                                                                     |
| 369  | wp.                                                                | Applies weakest-precondition reasoning to a program fragment.                                                                                     |
| 370  | call (: = {glob B} ==> = {glob B, res}).                           | Uses a procedure specification or adversary call rule inside a program logic proof.                                                               |
| 371  | + by sim.                                                          | Solves a proof branch produced by a previous split, transitivity, or case analysis.                                                               |
| 372  | by auto.                                                           | Discharges the current goal in one step using the cited simplification, lemma, or automation.                                                     |
| 373  | qed.                                                               | Ends the proof; EasyCrypt has accepted all remaining obligations.                                                                                 |
| 374  | <i>blank</i>                                                       | Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.                                                        |
| 375  | end section BimodalModuleSISReduction.                             | Closes the current section, discharging local module and variable scope.                                                                          |
| 376  | <i>blank</i>                                                       | Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.                                                        |
| 377  | end HAETAE_Assumptions.                                            | Closes the current theory or module block.                                                                                                        |